



HỆ THỐNG GỢI Ý PHIM DỰA TRÊN COLLABORATIVE FILTERING

Sử dụng MovieLens 100K

Môn học: **Trí tuệ nhân tạo**

Hướng tiếp cận: **Memory-Based CF + Model-Based CF**

Thuật toán: **User-Based CF, Item-Based CF, Matrix Factorization SVD**

Giao diện demo: **Streamlit Web App**

Sinh viên/Nhóm: **Nhóm 1**

GV Hướng dẫn: **TS. Bùi Mạnh Quân**

VẤN ĐỀ VÀ LÝ DO CHỌN ĐỀ TÀI

Bài toán đặt ra

- ▶ Người dùng có quá nhiều phim để lựa chọn (hàng nghìn đến hàng triệu nội dung).
- ▶ Khó tự tìm nội dung phù hợp với sở thích cá nhân.
- ▶ Hệ thống gợi ý giúp **cá nhân hóa trải nghiệm**.
- ▶ **Collaborative Filtering (CF)** tận dụng hành vi cộng đồng để dự đoán sở thích.
- ▶ MovieLens 100K là bộ dữ liệu chuẩn, phù hợp học thuật và kiểm thử thuật toán.

Nhiều phim + Nhiều người dùng



Dữ liệu đánh giá (Rating)



Mô hình gợi ý (Recommender)



Top-N phim phù hợp từng user

| DỮ LIỆU SỬ DỤNG: MOVIELENS 100K



100,000

Lượt đánh giá



943

Người dùng (Users)



1,682

Bộ phim (Movies)

Thang điểm: 1 – 5 sao. Mỗi user đánh giá ít nhất **20 ratings**.

Các file chính trong bộ dữ liệu:

- **u.data**: Chứa user_id, item_id, rating, timestamp (Dùng xây ma trận).
- **u.item**: Tên phim, ngày phát hành, IMDb URL, 19 thể loại.
- **u.user**: Thông tin nhân khẩu học (tuổi, giới tính, nghề nghiệp).

BIỂU DIỄN DỮ LIỆU: USER-ITEM MATRIX

Collaborative Filtering bắt đầu từ ma trận tương tác:

$$R \in \mathbb{R}^m \times n$$

- ▶ **m**: số người dùng (users).
- ▶ **n**: số phim (items).
- ▶ **R(u,i)**: rating của user *u* cho phim *i*.
- ▶ Ô chưa có dữ liệu được biểu diễn là 0 hoặc ?.

Ví dụ minh họa ma trận R:

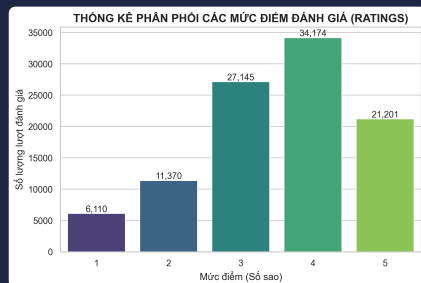
User / Movie	Titanic	Avatar	Matrix
U1	5	4	?
U2	5	5	4
U3	1	2	5

🎯 Bài toán chính: Dự đoán giá trị của các ô còn thiếu (?), từ đó chọn ra Top-N phim điểm cao nhất.

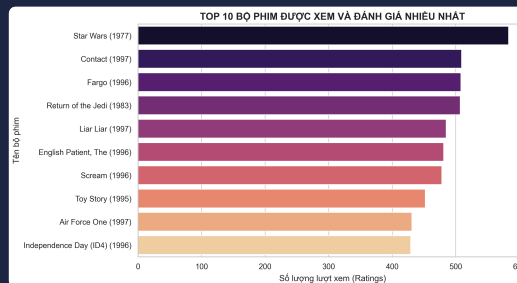
VẤN ĐỀ: DỮ LIỆU THƯA VÀ LONG-TAIL

- ▶ Không phải user nào cũng xem tất cả phim.
- ▶ Kích thước train matrix: **943 × 1682**.
- ▶ Độ thưa (Sparsity) train matrix: **94.96%**.
- ▶ Một số phim phổ biến có nhiều rating, phần lớn nằm ở **long-tail** (rất ít rating).

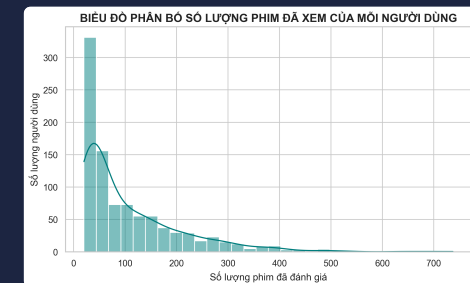
$$\text{Sparsity} = 1 - \frac{\text{Số rating thực tế}}{(\text{Users} \times \text{Items})}$$



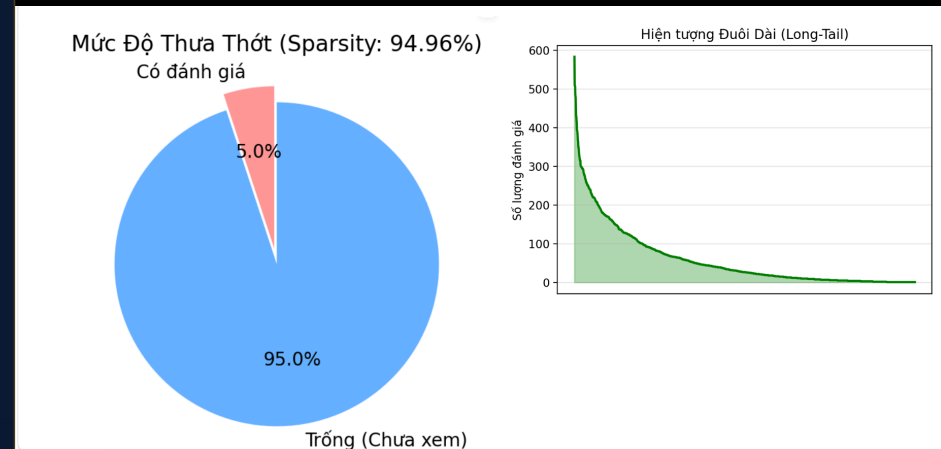
Rating Distribution (thường 3-4 sao)



Top Popular Movies (số ít chiếm ưu thế)



User Engagement (đánh giá không đồng đều)



Dữ liệu không đồng đều + ma trận thưa → Cần Collaborative Filtering để dự đoán.

COLLABORATIVE FILTERING LÀ GÌ?

? Làm gì?

- ▶ Dự đoán sở thích dựa trên **lịch sử rating của cộng đồng**.
- ▶ Không cần phân tích nội dung hay thể loại phim quá chi tiết.

⚙️ Làm như thế nào?

- ▶ Tìm user giống nhau hoặc item giống nhau.
- ▶ Hoặc học các "đặc trưng ẩn" từ ma trận.
- ▶ Dự đoán rating cho phim chưa xem.
- ▶ Sắp xếp và lấy Top-N phim cao nhất.

Phân loại thuật toán chính

Collaborative Filtering

- ├ Memory-Based CF
 - | └ User-Based CF
 - | └ Item-Based CF
- └ Model-Based CF
 - └ Matrix Factorization / SVD

KHI NÀO DÙNG THUẬT TOÁN NÀO?

Hướng tiếp cận	Nên dùng khi	Không phù hợp khi
User-Based CF	Số user vừa phải, cần giải thích bằng người dùng tương tự.	Số user quá lớn, cần tính toán realtime nhanh.
Item-Based CF	Item ổn định hơn user, gợi ý dựa trên "phim tương tự".	Item quá nhiều hoặc có item mới liên tục.
SVD / Matrix Fact.	Dữ liệu thưa, cần dự đoán nhanh sau huấn luyện.	Cần giải thích đơn giản logic, dữ liệu quá nhỏ.
Content-Based	Có user/item mới, có metadata phim (Cold-start).	Không có thông tin nội dung phim tốt.

Memory-Based: Dễ hiểu, giải thích

Model-Based: Mở rộng, đoán nhanh

Hybrid: Thực tế kết hợp

MEMORY-BASED CF: Ý TƯỞNG TỔNG QUÁT

Dùng trực tiếp ma trận rating để dự đoán.

Quy trình:

1. Xây dựng ma trận User-Item.
2. Tính độ tương đồng (Similarity).
3. Chọn K láng giềng gần nhất.
4. Dự đoán rating bằng trung bình có trọng số.
5. Sắp xếp và lấy Top-N.

Hai hướng chính:

- User-Based CF: *User giống User*
- Item-Based CF: *Phim giống Phim*

Đánh giá nhanh

✓ Ưu điểm:

- ▶ Dễ hiểu, dễ giải thích logic.
- ▶ Không cần huấn luyện mô hình phức tạp.
- ▶ Có thể triển khai nhanh.

✗ Nhược điểm:

- ▶ Tính toán chậm khi dữ liệu lớn.
- ▶ Nhạy cảm với dữ liệu thưa thớt.
- ▶ Gặp khó với Cold-start.

SIMILARITY: COSINE THUẦN TÚY

Đo góc giữa hai vector rating trong không gian.

$$\text{sim}(u,v) = (u \cdot v) / (||u|| \times ||v||)$$

$$\text{sim}(u,v) = \frac{\sum r(u,i)r(v,i)}{[\sqrt{\sum r(u,i)^2} \times \sqrt{\sum r(v,i)^2}]}$$

Trong đó:

- ▶ u, v : hai vector user/item.
- ▶ $r(u,i)$: rating của u cho i .

Ý nghĩa giá trị

- ▶ **sim = 1**: Rất giống nhau.
- ▶ **sim = 0**: Không liên hệ.
- ▶ **sim = -1**: Xu hướng đối lập.

Đánh giá

- ▶ **Ưu**: Đơn giản, tính nhanh (NumPy).
- ▶ **Nhược**: Chưa xử lý người chấm dễ/khó tính.

VẤN ĐỀ BIAS: NGƯỜI CHẤM DỄ, KHÓ, PHIM HOT

Vấn đề của Cosine thuần túy

- ▶ **Chấm dễ:** Thường cho 4–5 sao.
- ▶ **Chấm khó:** Thường cho 1–2 sao.
- ▶ **Phim Hot:** Được nhiều điểm cao do hiệu ứng đám đông.

Ví dụ trực quan:

User A: (5, 5, 5) → Dễ tính

User B: (1, 1, 1) → Khó tính

Cosine xem 2 vector này cùng hướng, nhưng thực tế mức độ yêu thích khác hẳn.

Cách nâng cấp giải quyết

- ▶ Với User–User: Dùng **Pearson Similarity**.
- ▶ Với Item–Item: Dùng **Adjusted Cosine Similarity**.
- ▶ Khi dự đoán Rating: Thêm **Mean / Biased Baseline**.

=> Trừ đi điểm trung bình hoặc cộng bias để đưa về cùng một hệ quy chiếu xu hướng.

USER-BASED CF: CÁCH HOẠT ĐỘNG

💡 Ý tưởng

Tìm những user có sở thích giống user mục tiêu, rồi dùng rating của họ để dự đoán phim user chưa xem.

☰ Quy trình

1. Chọn user mục tiêu u .
2. Tính similarity giữa u và các user khác.
3. Lọc các user đã đánh giá phim i .
4. Chọn $K = 40$ user gần nhất.
5. Dự đoán rating $\hat{r}(u, i)$.

Trong Code Dự Án

- ▶ Class: **UserBasedCollaborativeFiltering**
- ▶ Đo lường: **Pearson Similarity**
- ▶ K láng giềng: **40**
- ▶ Mode dự đoán: **KNN with Means** hoặc KNN with Biased Baseline.

CÔNG THỨC DỰ ĐOÁN **USER-BASED CF**

Công thức KNN with Means

$$\hat{r}(u,i) = \text{mean}(u) + \frac{\sum [\text{sim}(u,v) \times (r(v,i) - \text{mean}(v))]}{\sum |\text{sim}(u,v)|}$$

Trong đó:

- ▶ u : User mục tiêu.
- ▶ i : Phim cần dự đoán.
- ▶ v : User láng giềng đã đánh giá.
- ▶ $\text{mean}(u)$: Điểm TB của user u .
- ▶ $\text{sim}(u,v)$: Độ tương đồng.

Ý nghĩa cốt lõi:

- ▶ $r(v,i) - \text{mean}(v)$: Loại bỏ thói quen chấm lệch của láng giềng.
- ▶ $\text{sim}(u,v)$ làm trọng số: Càng giống ảnh hưởng càng lớn.
- ▶ Kết quả giới hạn 1-5 sao.

PEARSON SIMILARITY CHO USER-BASED CF

Bản chất: Cosine Similarity sau khi đã trừ đi trung bình rating của từng user.

$$\text{sim}(u,v) = \frac{\sum (r(u,i) - \text{mean}(u)) \times (r(v,i) - \text{mean}(v))}{\sqrt{\sum (r(u,i) - \text{mean}(u))^2} \times \sqrt{\sum (r(v,i) - \text{mean}(v))^2}}$$

Vì sao dùng Pearson?

- ▶ Loại bỏ ảnh hưởng người chấm dễ / chấm khó.
- ▶ Tập trung vào **xu hướng thích/không thích**.
- ▶ Phù hợp nhất với User-Based CF.
- ▶ Trong dự án: Tính bằng NumPy vectorization (nhANH).

ITEM-BASED CF: CÁCH HOẠT ĐỘNG

💡 Ý tưởng

Không tìm user giống user, mà tìm **Phim giống Phim**. Nếu user đã thích phim A, B thì hệ thống dự đoán user sẽ thích phim C (có tính chất giống A, B).

☰ Quy trình

1. Chọn user u và phim cần đoán i .
2. Tìm các phim j user u đã đánh giá.
3. Tính similarity giữa phim i và j .
4. Chọn $K = 40$ phim tương tự nhất.
5. Dự đoán từ các rating user đã cho.

Trong Code Dự Án

- ▶ Class: `ItemBasedCollaborativeFiltering`
- ▶ Đo lường: **Adjusted Cosine Similarity**
- ▶ Mode dự đoán: `Biased Baseline`.

CÔNG THỨC DỰ ĐOÁN ITEM-BASED CF

Item-Based KNN Cơ bản

$$\hat{r}(u,i) = \sum \text{sim}(i,j) \times r(u,j) / \sum |\text{sim}(i,j)|$$

- ▶ i: Phim cần dự đoán, j: Phim đã đánh giá.

Có Biased Baseline

$$b(u,i) = \mu + b_u + b_i$$

$$\hat{r}(u,i) = b(u,i) + \sum \text{sim}(i,j)(r(u,j) - b(u,j)) / \sum |\text{sim}(i,j)|$$

Ý nghĩa Biased Baseline:

- ▶ μ : Rating trung bình toàn hệ thống.
- ▶ b_u : Độ lệch user, b_i : Độ lệch phim.
- ▶ **Mục đích:** Giảm hoàn toàn ảnh hưởng nhiễu của phim quá hot hoặc user chấm điểm quá lệch.

ADJUSTED COSINE CHO ITEM-BASED CF

Dùng cho Item-Item Similarity.

$$\text{sim}(i,j) = \frac{\sum (r(u,i) - \text{mean}(u)) \times (r(u,j) - \text{mean}(u))}{\sqrt{\sum (r(u,i) - \text{mean}(u))^2} \times \sqrt{\sum (r(u,j) - \text{mean}(u))^2}}$$

- ▶ Hàm: `compute_adjusted_cosine...`
- ▶ Đầu vào: Matrix (Users × Items)
- ▶ Đầu ra: Matrix (Items × Items)

Vì sao cần Adjusted?

- ▶ Khi so sánh 2 phim, rating đến từ nhiều user khác nhau.
- ▶ Mỗi user có thói quen chấm điểm riêng.
- ▶ Trừ `mean(u)` giúp **loại bỏ bias của user**.
- ▶ Phù hợp hơn Cosine thuần túy cho item-item.

MODEL-BASED CF: MATRIX FACTORIZATION SVD

Học mô hình từ dữ liệu thay vì dùng trực tiếp toàn bộ ma trận rating.

- ▶ Phân rã ma trận rating thưa thành các ma trận đặc trưng ẩn (Latent factors).
- ▶ Mỗi user và phim được biểu diễn bằng vector k chiều.
- ▶ Dự đoán rating bằng tích vô hướng giữa vector user và vector phim.
- ▶ Trong dự án: $k = 20$ latent factors.

Công thức phân rã

$$R \approx P \times Q^T$$

R: Ma trận rating user-item.

P: Đặc trưng ẩn của user.

Q: Đặc trưng ẩn của item/phim.

CÔNG THỨC SVD CÓ BIAS

Công thức dự đoán rating

$$\hat{r}(u,i) = \mu + b_u + b_i + P_u \cdot Q_i$$

- ▶ μ : Rating TB toàn hệ thống.
- ▶ b_u : Bias của user u .
- ▶ b_i : Bias của phim i .
- ▶ $P_u \cdot Q_i$: Mức độ phù hợp giữa sở thích ẩn của user và đặc trưng ẩn của phim.

Tham số trong dự án

Tham số	Giá trị
Số latent factors (k)	20
Learning rate (γ)	0.005
Regularization (λ)	0.02
Epochs	20

HUẤN LUYỆN SVD: THUẬT TOÁN SGD

Stochastic Gradient Descent

$$e(u,i) = r(u,i) - \hat{r}(u,i)$$

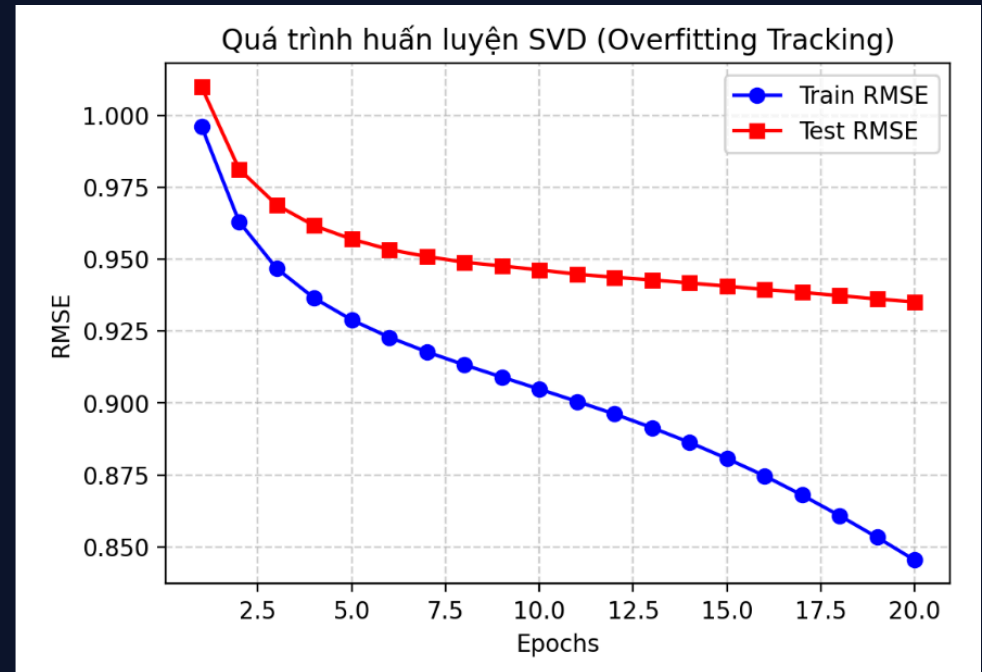
$$b_u \leftarrow b_u + \gamma(e_{ui} - \lambda b_u)$$

$$b_i \leftarrow b_i + \gamma(e_{ui} - \lambda b_i)$$

$$P_u \leftarrow P_u + \gamma(e_{ui} Q_i - \lambda P_u)$$

$$Q_i \leftarrow Q_i + \gamma(e_{ui} P_u - \lambda Q_i)$$

- ▶ γ : Learning rate.
- ▶ λ : Regularization chống overfitting.



ĐÁNH GIÁ THUẬT TOÁN: MAE VÀ RMSE

Mean Absolute Error (MAE)

$$MAE = (1/N) \sum |r(u,i) - \hat{r}(u,i)|$$

Sai số tuyệt đối trung bình, dễ diễn giải (vd lệch 0.7 sao).

Root Mean Square Error (RMSE)

$$RMSE = \sqrt{ (1/N) \sum (r(u,i) - \hat{r}(u,i))^2 }$$

Phạt nặng hơn các lỗi dự đoán lớn.



Tiêu chí

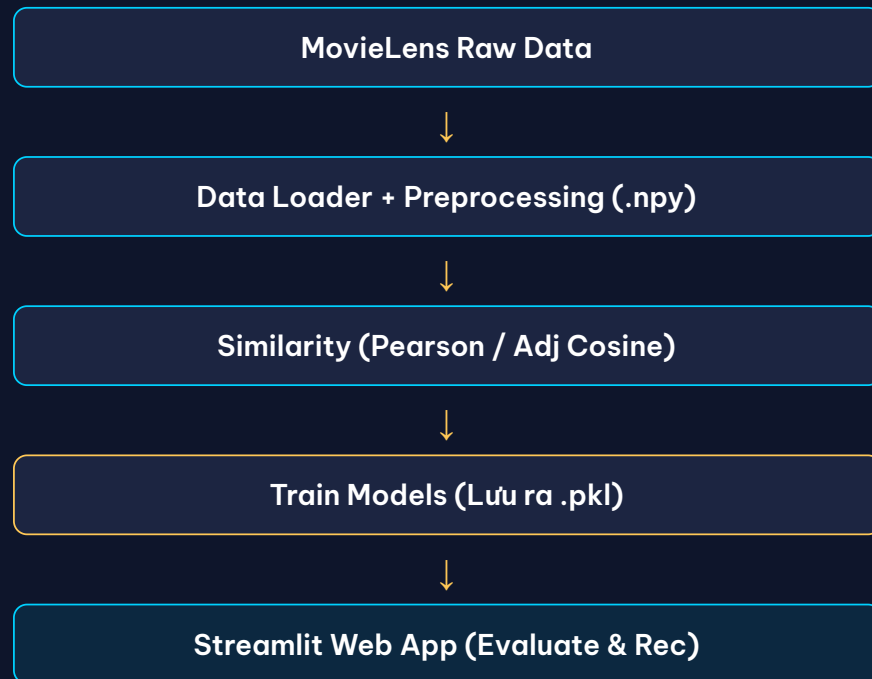
**Cả 2 chỉ số
CÀNG THẤP CÀNG TỐT**

Thang điểm 1-5, sai số dưới ~1 sao là mức tương đối tốt.

CẤU TRÚC DỰ ÁN (README.MD)

```
movie_recommendation_system/
├── data/
│   ├── raw/           # MovieLens 100K gốc
│   └── processed/      # train_matrix.npy, test_matrix.npy
├── models/            # file trọng số .pkl sau huấn luyện
├── notebooks/         # EDA và kiểm chứng thư viện
├── scripts/
│   └── train_pipeline.py # pipeline huấn luyện toàn bộ mô hình
├── src/
│   ├── data_loader.py  # đọc dữ liệu, tạo ma trận
│   ├── similarity.py   # Cosine, Pearson, Adjusted Cosine
│   ├── recommender.py  # User-Based, Item-Based, SVD
│   ├── content_based.py # TF-IDF Content-Based fallback
│   └── evaluation.py   # MAE/RMSE
├── app.py             # giao diện Streamlit
└── main.py            # kiểm thử terminal
```

PIPELINE SẢN PHẨM CODE



Điểm kỹ thuật nổi bật

- ▶ **Cache ma trận:** Lưu bằng .npy giúp load cực nhanh.
- ▶ **Offline Training:** Train mô hình trước, lưu trọng số bằng pickle (chỉ cần chạy predict trên web).
- ▶ **Batch Prediction:** Dùng vector hóa NumPy tăng tốc hàm MAE/RMSE.
- ▶ **Streamlit 3 Tabs:** Trực quan hóa, Gợi ý, Đánh giá.

CÁC THUẬT TOÁN ĐÃ TRIỂN KHAI TRONG SẢN PHẨM

Thuật toán	Nhóm	Vai trò trong sản phẩm
User-Based CF	Memory-Based	Tìm user tương tự, dự đoán bằng KNN.
Item-Based CF	Memory-Based	Tìm phim tương tự, dự đoán bằng KNN.
Matrix Factorization SVD	Model-Based	Học latent factors, dự đoán nhanh.
Content-Based TF-IDF	Content-Based	Gợi ý phim tương tự, hỗ trợ cold-start item.

Trọng tâm báo cáo: So sánh hiệu năng giữa User-Based, Item-Based và SVD.

THUẬT TOÁN ƯU TIÊN TRONG SẢN PHẨM

Ưu tiên Demo: SVD

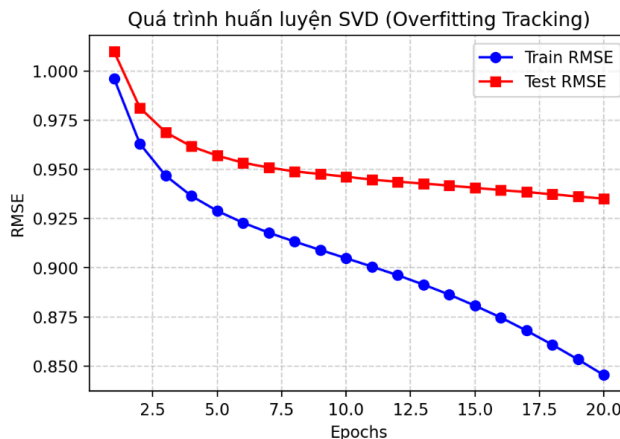
- ▶ Phù hợp nhất với ma trận rating rất thưa.
- ▶ Sau khi train, **dự đoán cực nhanh** bằng tích vô hướng vector.
- ▶ Mô hình lưu siêu nhẹ trong `svd_weights.pkl`.
- ▶ Phù hợp hơn nếu mở rộng hệ thống thực tế.

Thực tế kết quả

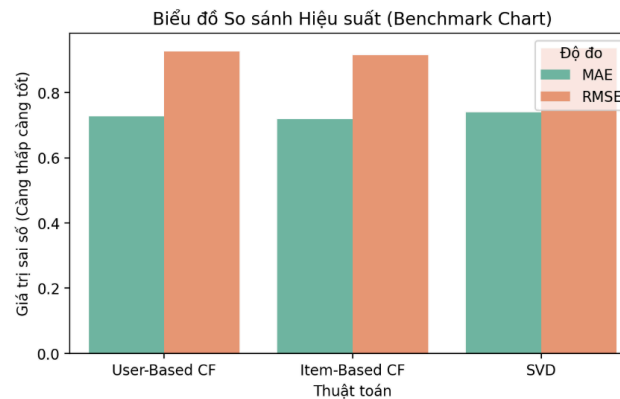
- ▶ **Item-Based / User-Based** có thể đạt MAE/RMSE thấp hơn nhẹ trên tập 100K.
- ▶ Lý do: Bộ dữ liệu 100K ở mức vừa/nhỏ, Memory-Based vẫn hoạt động rất hiệu quả.
- ▶ **Giải pháp Web:** Cho phép chọn cả 3 thuật toán để tự do so sánh kết quả.

KẾT QUẢ SO SÁNH THUẬT TOÁN

So Sánh Độ Lệch Sai Số Giữa Các Thuật Toán Lọc Cộng Tác



Chạy So Sánh Kép Hệ Thống



Thuật toán	MAE	RMSE
0 User-Based CF	0.7262	0.9261
1 Item-Based CF	0.7188	0.9145
2 Matrix Factorization (SVD)	0.7397	0.9351

Thuật toán	RMSE	MAE	Nhận xét
User-Based CF	0.9261	0.7262	Dễ giải thích, kết quả tốt
Item-Based CF	0.9146	0.7189	Tốt nhất hiện tại
SVD	0.9362	0.7407	Dự đoán nhanh, dễ mở rộng

Kết luận: Item-Based CF đạt sai số thấp nhất, thể hiện độ ổn định trên tập MovieLens. SVD nhỉnh hơn về tốc độ triển khai online thực tế.

ĐỐI CHIẾU VỚI THƯ VIỆN (SCIKIT-SURPRISE)

Kết quả từ thư viện tham khảo

Mô hình thư viện	RMSE	MAE
KNNBaseline User-Based	0.9197	0.7190
KNNBaseline Item-Based	0.9169	0.7188

Trích xuất từ notebook: 02_library_verification.ipynb

Ý nghĩa việc kiểm chứng

- ▶ Kết quả tự cài đặt có xu hướng **gần sát** với thư viện chuẩn.
- ▶ Xác nhận công thức toán học và logic thuật toán tự cài là hợp lý.
- ▶ Sai số không lệch quá xa so với baseline phổ biến của thế giới.

TỔNG HỢP BIỂU ĐỒ PHÂN TÍCH

Biểu đồ EDA (Notebooks)

- ▶ **Rating Distribution:** Phân phối điểm 1–5.
- ▶ **Top 10 Popular Movies:** Hiện tượng phim phổ biến.
- ▶ **User Engagement:** Số lượt rating mỗi user.

Biểu đồ App (Streamlit)

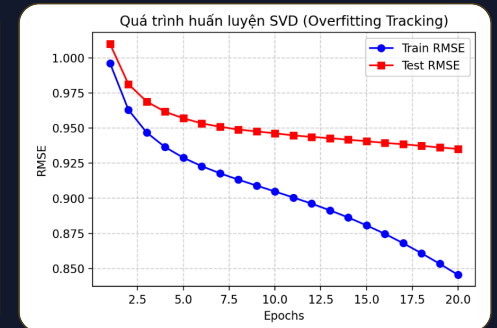
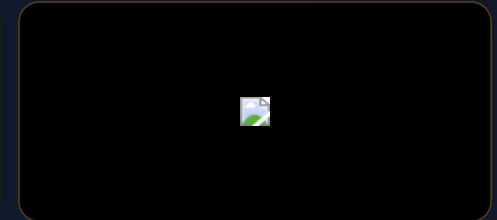
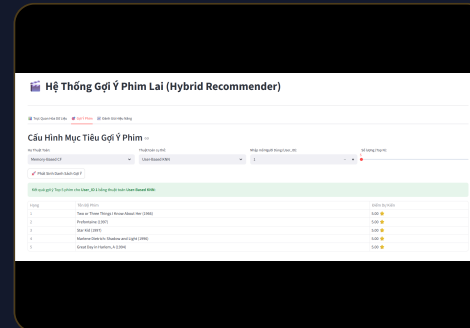
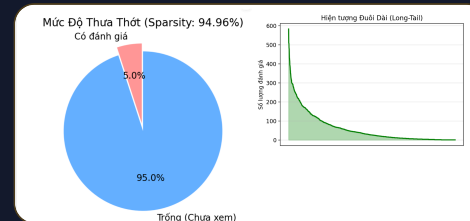
- ▶ **Sparsity Pie & Long-Tail:** Độ thưa dữ liệu.
- ▶ **MAE/RMSE Comparison:** So sánh độ lệch.
- ▶ **SVD Training Curve:** Train/Test RMSE theo Epochs.

GIAO DIỆN SẢN PHẨM (STREAMLIT WEB APP)

Ứng dụng Web gồm 3 Tabs:

- ▶ **Tab 1 - Trực quan:** Độ thưa, Long-Tail, Không gian PCA 2D SVD.
- ▶ **Tab 2 - Gợi ý:** Chọn model (SVD, User/Item CF), nhập User ID, xem Top-N.
- ▶ **Tab 3 - Đánh giá:** Train curve SVD, So sánh thuật toán.

Lệnh chạy: `streamlit run app.py`



VÍ DỤ GỢI Ý TOP-N PHIM (USER 1)

Hệ Thống Gợi Ý Phim Lai (Hybrid Recommender)

Trực Quan Hóa Dữ Liệu Gợi Ý Phim Đánh Giá Hiệu Năng

Cấu Hình Mục Tiêu Gợi Ý Phim

Họ Thuật Toán:

Model-Based CF

Thuật toán cụ thể:

SVD

Nhập mã Người Dùng (User_ID):

1

Số lượng (Top N):

5

Phát Sinh Danh Sách Gợi Ý

Kết quả gợi ý Top 5 phim cho User_ID 1 bằng thuật toán SVD:

Hạng	Tên Bộ Phim	Điểm Dự Kiến
1	One Flew Over the Cuckoo's Nest (1975)	4.72 ★
2	Schindler's List (1993)	4.68 ★
3	Great Escape, The (1963)	4.65 ★
4	Close Shave, A (1995)	4.62 ★
5	Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1963)	4.57 ★

Luồng sinh gợi ý:

- Lọc các phim user **chưa xem**.
- Dự đoán rating cho từng phim bằng SVD.
- Sắp xếp giảm dần theo điểm dự đoán.
- Trả về Top 5 phim cao điểm nhất hiển thị ra UI.

Hạng	Tên phim	Dự đoán
1	Close Shave, A (1995)	4.63 ★
2	Blade Runner (1982)	4.59 ★
3	Casablanca (1942)	4.58 ★
4	One Flew Over the Cuckoo's (1975)	4.55 ★
5	Sunset Blvd. (1950)	4.54 ★

NHẬN XÉT ƯU/NHƯỢC ĐIỂM

Thuật toán	Ưu điểm	Nhược điểm
User-Based	Dễ hiểu, giải thích bằng user tương tự.	Chậm khi số user lớn, nhạy với sparsity.
Item-Based	Ổn định, kết quả rất tốt trên 100K.	Cần tạo ma trận item-item, khó với item mới.
SVD	Đoán nhanh, học latent factor, cực tốt với data thưa.	Khó giải thích trực quan, cần huấn luyện lại.
Content-Based	Giải quyết cold-start item, tận dụng metadata.	Gợi ý bị giới hạn, kém đa dạng.

Chiến lược chọn: Chính xác cao -> Item-Based | Dễ giải thích -> User-Based | Tốc độ/Mở rộng -> SVD

HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

⚠ Hạn chế hiện tại

- ▶ Dữ liệu siêu thưa, nhiều cặp chưa có rating.
- ▶ Vấn đề **Cold-start** nghiêm trọng (User mới / Phim mới).
- ▶ Chưa khai thác hết metadata (genre, age, occupation).
- ▶ Kết quả dự đoán còn phụ thuộc nhiều vào Hyperparameters.

🔮 Hướng phát triển

- ▶ Xây dựng **Hybrid Recommender** kết hợp CF + Content-Based.
- ▶ Tối ưu siêu tham số bằng Grid Search / Random Search.
- ▶ Dùng 5-fold cross validation đánh giá vững chắc hơn.
- ▶ Thêm chức năng Online Learning (Nhập rating mới trực tiếp).

KẾT LUẬN

- ▶ Đã xây dựng thành công hệ thống gợi ý trên **MovieLens 100K**.
- ▶ Đã triển khai hai hướng chính của Collaborative Filtering:
 - ▶ **Memory-Based CF**: User-Based, Item-Based.
 - ▶ **Model-Based CF**: Matrix Factorization SVD.
- ▶ Đã cài đặt thành công similarity chống nhiễu (**Pearson, Adjusted Cosine**).
- ▶ Đã so sánh đo lường **MAE / RMSE** độc lập.
- ▶ Hoàn thiện đóng gói giao diện **Streamlit** demo trực quan.

Collaborative Filtering là cốt lõi

Kết hợp đa thuật toán giúp hệ thống cân bằng giữa chính xác, tốc độ và linh hoạt thực tế.

TÀI LIỆU THAM KHẢO

1. F. Maxwell Harper and Joseph A. Konstan. 2015. *The MovieLens Datasets: History and Context*. ACM Transactions on Interactive Intelligent Systems.
2. GroupLens. **MovieLens 100K Dataset**. <https://grouplens.org/datasets/movielens/100k/>
3. Ricci, F., Rokach, L., Shapira, B. *Recommender Systems Handbook*. Springer.
4. NumPy Documentation: <https://numpy.org/doc/>
5. Pandas Documentation: <https://pandas.pydata.org/docs/>
6. Streamlit Documentation: <https://docs.streamlit.io/>
7. Scikit-surprise Documentation: <https://surpriselib.com/>

XIN CẢM ƠN!

Q&A - Mời thầy cô và các bạn đặt câu hỏi